

# РЕФЕРАТ

---

*по дисциплине: «Операционные системы»*

**Тема: «Сравнительный анализ алгоритмов замещения страниц»**

Выполнила: Макаренко Александра Александровна

Группа: НКАбд-07-25

2026

## Содержание

|                                                          |   |
|----------------------------------------------------------|---|
| Введение .....                                           | 3 |
| 1. Принципы замещения страниц в виртуальной памяти ..... | 4 |
| 2. Алгоритм OPT (Optimal) — теоретический эталон .....   | 4 |
| 3. Алгоритм FIFO (First In, First Out) .....             | 5 |
| 4. Алгоритм NRU (Not Recently Used) .....                | 5 |
| 5. Алгоритм LRU (Least Recently Used) .....              | 6 |
| 6. Алгоритм NFU (Not Frequently Used) .....              | 7 |
| 7. Сравнительный анализ алгоритмов .....                 | 7 |
| Заключение .....                                         | 8 |
| Список использованных источников .....                   | 9 |

## Введение

Управление памятью является одной из ключевых задач операционных систем. В современных вычислительных системах объём оперативной памяти всегда ограничен, тогда как потребности запущенных процессов могут многократно его превышать. Механизм виртуальной памяти позволяет решить это противоречие: каждый процесс получает собственное логическое адресное пространство, которое отображается на физические страницы по мере необходимости.

Когда процессу требуется страница, которой нет в оперативной памяти, происходит так называемое страничное прерывание (page fault). Для загрузки нужной страницы операционная система должна освободить один из занятых фреймов памяти — вытеснив из него другую страницу. От того, насколько грамотно выбирается жертва, напрямую зависит производительность всей системы: неудачный выбор приводит к лавинообразному росту страничных прерываний, а при крайне неблагоприятном стечении обстоятельств — к явлению «пробуксовки» (thrashing), когда система тратит большую часть времени на подкачку страниц, а не на полезную работу.

Алгоритм замещения страниц — это стратегия, которую использует операционная система для выбора вытесняемой страницы. Существует несколько классических алгоритмов: от теоретически оптимального OPT до практически простого FIFO. Каждый из них представляет собой компромисс между сложностью реализации, накладными расходами и эффективностью работы.

Целью настоящего реферата является систематизированное рассмотрение и сравнительный анализ основных алгоритмов замещения страниц: OPT, FIFO, NRU, LRU и NFU. Для каждого алгоритма описывается принцип действия, перечисляются достоинства и недостатки, после чего приводится сводная сравнительная таблица.

## 1. Принципы замещения страниц в виртуальной памяти

Виртуальная память разбивается на страницы фиксированного размера (обычно 4 КБ). Физическая оперативная память делится на фреймы того же размера. Таблица страниц каждого процесса хранит соответствие между номерами виртуальных страниц и номерами физических фреймов.

Когда процесс обращается к адресу, чья страница отсутствует в памяти, аппаратура генерирует прерывание. Обработчик прерывания выполняет следующие действия: если есть свободный фрейм — загружает нужную страницу с диска; если свободных фреймов нет — выбирает страницу-жертву, при необходимости записывает её на диск (если она была изменена) и загружает на освободившееся место нужную страницу.

Бит R (Referenced) устанавливается аппаратурой при любом обращении к странице. Бит M (Modified) устанавливается при записи. Операционная система периодически сбрасывает бит R, что позволяет отслеживать «недавнюю» активность страниц. Многие алгоритмы замещения опираются именно на эти биты.

## 2. Алгоритм OPT (Optimal)

Алгоритм OPT, предложенный Ладлоу Беладии в 1966 году, представляет собой теоретически оптимальную стратегию: для замещения всегда выбирается та страница, к которой следующее обращение произойдёт позже всего (или не произойдёт вовсе). Доказано, что ни один другой детерминированный алгоритм замещения не может дать меньшее число страничных прерываний при той же последовательности обращений.

### Достоинства

- Минимально возможное число страничных прерываний для данной последовательности обращений.
- Служит «золотым стандартом» — эталоном для оценки эффективности всех остальных алгоритмов.
- Не подвержен аномалии Беладии: добавление фреймов никогда не увеличивает число прерываний.

### Недостатки

- Принципиально нереализуем в реальных системах: операционная система не может знать будущее поведение программы и последовательность предстоящих обращений к памяти.
- Применяется исключительно в исследовательских целях — для сравнения и оценки других алгоритмов путём моделирования на трассах обращений.

### 3. Алгоритм FIFO (First In, First Out)

FIFO — простейший алгоритм замещения. Страницы упорядочиваются в очередь в порядке их загрузки в память. При необходимости освободить фрейм всегда вытесняется страница, стоящая в голове очереди, то есть та, которая находится в памяти дольше всего.

#### Достоинства

- Исключительная простота реализации: требуется лишь поддержка кольцевой очереди указателей на фреймы.
- Практически нулевые накладные расходы по памяти и процессорному времени.

#### Недостатки

- Низкая эффективность: нахождение страницы в памяти дольше всего совсем не означает, что она используется редко — алгоритм игнорирует реальный характер обращений.
- Аномалия Беладии: парадоксальный эффект, при котором увеличение числа доступных фреймов может привести к росту, а не снижению числа страничных прерываний. Это делает поведение алгоритма трудно предсказуемым.

### 4. Алгоритм NRU (Not Recently Used)

Алгоритм NRU (не использовавшийся в последнее время) эксплуатирует аппаратные биты состояния страниц. На основе значений битов R и M каждая страница относится к одному из четырёх классов:

- Класс 0:  $R = 0, M = 0$  — страница не читалась и не изменялась; наиболее приоритетна для вытеснения.
- Класс 1:  $R = 0, M = 1$  — страница не читалась, но была изменена.
- Класс 2:  $R = 1, M = 0$  — страница читалась, но не изменялась.
- Класс 3:  $R = 1, M = 1$  — страница и читалась, и изменялась; наименее приоритетна для вытеснения.

При необходимости замещения выбирается случайная страница из непустого класса с наименьшим номером. Операционная система периодически (например, по прерыванию таймера) сбрасывает бит R у всех страниц, что обеспечивает учёт именно «недавних» обращений.

#### Достоинства

- Быстрая реализация: не требует сложных структур данных — достаточно аппаратных битов.
- Учитывает сразу два параметра: активность обращений и факт модификации страницы.
- Работает лучше FIFO и не подвержен аномалии Беладии.

#### **Недостатки**

- Менее точен, чем LRU: алгоритм различает лишь четыре класса, не учитывая порядок обращений внутри каждого класса.
- Выбор страницы внутри класса случаен, что может приводить к субоптимальным решениям.
- Требуется регулярного сброса бита R операционной системой.

## 5. Алгоритм LRU (Least Recently Used)

Алгоритм LRU основан на принципе темпоральной локальности: если страница давно не использовалась, вероятность обращения к ней в ближайшем будущем невелика. При вытеснении выбирается страница, к которой не было обращений наибольшее время.

Точная реализация LRU требует фиксировать момент последнего обращения к каждой странице. На практике используются два основных подхода: связный список, в котором страница перемещается в голову при каждом обращении (вытесняется страница из хвоста), либо матрица  $n \times n$ , где при каждом обращении к странице  $i$  устанавливаются все биты строки  $i$  и сбрасываются все биты столбца  $i$  — страница с наименьшим значением строки является наиболее давно использованной.

### Достоинства

- Высокая эффективность: по числу страничных прерываний результаты LRU близки к оптимальным (OPT).
- Принадлежит к классу «стековых алгоритмов», что теоретически гарантирует отсутствие аномалии Беладии.

### Недостатки

- Значительные накладные расходы: каждое обращение к памяти требует обновления списка или матрицы, что создаёт серьёзную нагрузку на систему при больших объёмах памяти.
- Сложность аппаратной поддержки: для обновления структур данных при каждом обращении требуется либо специализированная аппаратура, либо программные приближения.

## 6. Алгоритм NFU (Not Frequently Used)

Алгоритм NFU является программным приближением LRU. Для каждой страницы ведётся счётчик обращений. При каждом прерывании таймера операционная система добавляет значение бита  $R$  к счётчику соответствующей страницы, после чего сбрасывает бит  $R$ . При необходимости вытеснения выбирается страница с наименьшим значением счётчика.

Существует улучшенная версия алгоритма — «состаривание» (Aging): перед добавлением бита  $R$  счётчик сдвигается вправо на один бит, что реализует



экспоненциальное «забывание» старых обращений и значительно улучшает адаптивность алгоритма.

### **Достоинства**

- Проще в реализации по сравнению с LRU: нет необходимости хранить точный порядок всех обращений.
- Небольшие накладные расходы при каждом обращении к памяти — обновление происходит лишь по прерыванию таймера.

### **Недостатки**

- «Плохая память»: страница, которая была очень популярна в прошлом, накопила большой счётчик и будет долго «защищена» от вытеснения, даже если больше не используется.
- Низкая адаптивность к смене фаз работы программы (без модификации с «состариванием»).

## 7. Сравнительный анализ алгоритмов

В таблице 1 представлено сводное сравнение рассмотренных алгоритмов замещения страниц по ключевым характеристикам: практической реализуемости, «интеллекту» (учёту паттернов обращений), производительности и подверженности аномалии Белادي.

Таблица 1 — Сравнение алгоритмов замещения страниц

| Алгоритм | Реализуемость | Интеллект    | Производительность | Аномалия Белادي | Примечание |
|----------|---------------|--------------|--------------------|-----------------|------------|
| OPT      | Нереализуем   | Да (будущее) | Наилучшая          | Нет             | Эталон     |
| LRU      | Сложная       | Да (прошрое) | Высокая            | Нет             | Стековый   |
| NFU      | Средняя       | Частично     | Средняя            | Нет             |            |
| NRU      | Простая       | Частично     | Выше FIFO          | Нет             |            |
| FIFO     | Простая       | Нет          | Низкая             | Есть            | Проще всех |

Из таблицы видно, что алгоритмы образуют своеобразный спектр. На одном конце находится OPT — недостижимый идеал, задающий нижнюю границу числа страничных прерываний. На другом — FIFO, чья простота достигается ценой низкой эффективности и непредсказуемой аномалии Белادي. LRU занимает выгодное положение: высокая эффективность без аномалии, однако ценой значительных накладных расходов. NRU и NFU представляют собой практические компромиссы между сложностью и качеством замещения.

На практике в большинстве современных операционных систем применяются вариации алгоритма «часов» (Clock) — эффективные аппроксимации LRU, совмещающие приемлемую производительность с умеренными накладными расходами.

## **Заключение**

В ходе работы были рассмотрены пять классических алгоритмов замещения страниц: OPT, FIFO, NRU, LRU и NFU. Каждый из них воплощает определённую стратегию выбора страницы-жертвы при возникновении страничного прерывания.

Алгоритм OPT является теоретическим оптимумом, однако нереализуем на практике ввиду невозможности предсказать будущее поведение программы. Он служит эталоном для оценки других алгоритмов. FIFO, несмотря на предельную простоту, страдает низкой эффективностью и подвержен аномалии Беладии, что ограничивает его применимость. NRU и NFU представляют собой практические компромиссы: они работают значительно лучше FIFO, но уступают LRU по качеству принимаемых решений. LRU демонстрирует наилучшую практическую эффективность среди реализуемых алгоритмов, однако требует существенных вычислительных ресурсов для поддержания точной информации об истории обращений.

Таким образом, выбор алгоритма замещения страниц всегда представляет собой компромисс между эффективностью замещения, сложностью реализации и накладными расходами. Понимание этих компромиссов является необходимой основой для проектирования и настройки операционных систем в соответствии с требованиями конкретных приложений и аппаратных платформ.

### **Список использованных источников**

- Таненбаум Э., Бос Х. Современные операционные системы. — 4-е изд. — СПб.: Питер, 2015. — 1120 с.
- Silberschatz A., Galvin P. B., Gagne G. Operating System Concepts. — 10th ed. — Wiley, 2018. — 944 p.
- Belady L. A. A study of replacement algorithms for a virtual-storage computer // IBM Systems Journal. — 1966. — Vol. 5, No. 2. — P. 78–101.
- Stallings W. Operating Systems: Internals and Design Principles. — 9th ed. — Pearson, 2018. — 816 p.